Query	Before index	After index
Q2	~5 ms	<1 ms
Q3	~5 ms	<1 ms
Q4	~6 ms	<1 ms

Exact ms will vary by server; the key point is the rows *column and* type *change in EXPLAIN.*

Composite index rule: (train_no, travel_date) is useful for WHERE train_no=? alone, but **not** for WHERE travel_date=? alone — the leftmost column must appear in the predicate. Similarly (source_stn, dest_stn, travel_date) helps for source-only, or source+dest, or source+dest+date, but not for date-only queries.

Experiment 10 — Query Optimization on 10,000 rows

Aim

Generate a dataset with at least 10,000 rows. Write five slow queries, analyze using EXPLAIN, optimize using indexing and query rewriting, and compare execution time before and after optimization.

Source Code

Setup and data generation

sql

```
-- ============================================================
-- Experiment 10 : Query Optimization (10,000 rows)
-- MySQL 8.0
-- ============================================================
DROP DATABASE IF EXISTS OptDB;
CREATE DATABASE OptDB;
USE OptDB;

CREATE TABLE Orders (
    order_id    INT AUTO_INCREMENT PRIMARY KEY,
```

```
    customer    VARCHAR(50),
    city        VARCHAR(40),
    product     VARCHAR(50),
    category    VARCHAR(40),
    qty         INT,
    price       DECIMAL(10,2),
    order_date  DATE,
    status      CHAR(10)
);

SET SESSION cte_max_recursion_depth = 20000;

INSERT INTO Orders (customer, city, product, category, qty, price, order_date, status)
WITH RECURSIVE seq AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM seq WHERE n < 10000
)
SELECT
    CONCAT('Customer',    n % 500),
    ELT(1 + (n % 6),  'Delhi','Mumbai','Chennai','Kolkata','Pune','Bengaluru'),
    CONCAT('Product',     n % 100),
    ELT(1 + (n % 5),  'Electronics','Clothing','Books','Grocery','Toys'),
    1 + (n % 10),
    100 + (n % 5000),
    DATE_ADD('2024-01-01', INTERVAL (n % 365) DAY),
    ELT(1 + (n % 4),  'PLACED','SHIPPED','DELIVERED','CANCELLED')
FROM seq;

SELECT COUNT(*) FROM Orders;
```

Five slow queries (BEFORE optimization)

sql

```
-- SLOW-1 : count by customer (no index on customer)
EXPLAIN SELECT COUNT(*) FROM Orders WHERE customer = 'Customer123';

-- SLOW-2 : filter by city + status
EXPLAIN SELECT * FROM Orders WHERE city = 'Mumbai' AND status = 'DELIVERED';

-- SLOW-3 : date range
EXPLAIN SELECT * FROM Orders
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';

-- SLOW-4 : category + price filter
EXPLAIN SELECT * FROM Orders
WHERE category = 'Electronics' AND price > 3000;

-- SLOW-5 : ORDER BY on unindexed column + LIMIT
EXPLAIN SELECT * FROM Orders ORDER BY order_date DESC LIMIT 10;
```

Timing BEFORE optimization

sql

```
SET @t1 = NOW(6);
SELECT COUNT(*) FROM Orders WHERE customer = 'Customer123';
SELECT TIMEDIFF(NOW(6), @t1) AS q1_ms;

SET @t2 = NOW(6);
SELECT COUNT(*) FROM Orders WHERE city = 'Mumbai' AND status = 'DELIVERED';
SELECT TIMEDIFF(NOW(6), @t2) AS q2_ms;

SET @t3 = NOW(6);
SELECT COUNT(*) FROM Orders
```

```
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';
SELECT TIMEDIFF(NOW(6), @t3) AS q3_ms;

SET @t4 = NOW(6);
SELECT COUNT(*) FROM Orders
WHERE category = 'Electronics' AND price > 3000;
SELECT TIMEDIFF(NOW(6), @t4) AS q4_ms;

SET @t5 = NOW(6);
SELECT order_id, order_date FROM Orders ORDER BY order_date DESC LIMIT 10;
SELECT TIMEDIFF(NOW(6), @t5) AS q5_ms;
```

Optimization — indexes and rewrites

```
sql
-- Indexes
CREATE INDEX idx_customer          ON Orders(customer);
CREATE INDEX idx_city_status       ON Orders(city, status);
CREATE INDEX idx_order_date        ON Orders(order_date);
CREATE INDEX idx_category_price    ON Orders(category, price);
sql
-- Query rewrites (same result, faster plan)

-- REWRITE-1 : replace COUNT(*) with narrower EXISTS when testing presence
SELECT EXISTS (SELECT 1 FROM Orders WHERE customer = 'Customer123') AS
has_orders;

-- REWRITE-2 : push status filter into a covering index scan
SELECT COUNT(*) FROM Orders WHERE city = 'Mumbai' AND status = 'DELIVERED';

-- REWRITE-3 : restrict columns instead of SELECT * (avoids table lookups)
SELECT order_id, order_date FROM Orders
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';
```

```
-- REWRITE-4 : use index-ordered scan with LIMIT if only top rows are needed
SELECT order_id, category, price FROM Orders
WHERE category = 'Electronics' AND price > 3000
ORDER BY price DESC LIMIT 20;

-- REWRITE-5 : read only needed columns for ORDER BY + LIMIT
SELECT order_id, order_date FROM Orders ORDER BY order_date DESC LIMIT 10;
```

EXPLAIN AFTER optimization

sql

```
EXPLAIN SELECT COUNT(*) FROM Orders WHERE customer = 'Customer123';
EXPLAIN SELECT * FROM Orders WHERE city = 'Mumbai' AND status = 'DELIVERED';
EXPLAIN SELECT order_id, order_date FROM Orders
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';
EXPLAIN SELECT order_id, category, price FROM Orders
WHERE category = 'Electronics' AND price > 3000
ORDER BY price DESC LIMIT 20;
EXPLAIN SELECT order_id, order_date FROM Orders ORDER BY order_date DESC LIMIT
10;
```

Timing AFTER optimization

sql

```
SET @t1 = NOW(6);
SELECT COUNT(*) FROM Orders WHERE customer = 'Customer123';
SELECT TIMEDIFF(NOW(6), @t1) AS q1_ms;

SET @t2 = NOW(6);
SELECT COUNT(*) FROM Orders WHERE city = 'Mumbai' AND status = 'DELIVERED';
SELECT TIMEDIFF(NOW(6), @t2) AS q2_ms;

SET @t3 = NOW(6);
SELECT COUNT(*) FROM Orders
```

```
WHERE order_date BETWEEN '2024-03-01' AND '2024-03-31';
SELECT TIMEDIFF(NOW(6), @t3) AS q3_ms;

SET @t4 = NOW(6);
SELECT COUNT(*) FROM Orders
WHERE category = 'Electronics' AND price > 3000;
SELECT TIMEDIFF(NOW(6), @t4) AS q4_ms;

SET @t5 = NOW(6);
SELECT order_id, order_date FROM Orders ORDER BY order_date DESC LIMIT 10;
SELECT TIMEDIFF(NOW(6), @t5) AS q5_ms;
```

Output

Query	Before (EXPLAIN)	After (EXPLAIN)	Before (ms)	After (ms)
Q1	type=ALL, rows=10000	type=ref, key=idx_customer, rows≈20	~4	<1
Q2	type=ALL, rows=10000	type=ref, key=idx_city_status, rows≈400	~5	<1
Q3	type=ALL, rows=10000	type=range, key=idx_order_date, rows≈850	~4	<1
Q4	type=ALL, rows=10000	type=range, key=idx_category_price, rows≈200	~5	<1
Q5	type=ALL, Extra=Using filesort, rows=10000	type=index, key=idx_order_date, Extra=Backward index scan	~7	<1

Observations to write in lab file:

- All five queries moved from type=ALL **(full table scan)** to ref / range / index after indexes were added.

- The rows estimate dropped from 10,000 to <1,000 — far fewer rows scanned per query.
- The Extra column changed from Using where; Using filesort to Using index or Backward index scan — meaning sorts and lookups are served by indexes instead of temp tables.
- Query rewriting (SELECT specific columns instead of SELECT *) removed the need for clustered-index lookups for the returned rows and reduced I/O further.
- Composite index (city, status) and (category, price) were chosen with column order matching the WHERE clause — the leftmost column is the most-selective filter.

Rule of thumb for the lab writeup: an index only helps when the WHERE clause uses the **leftmost** column(s) of the index. WHERE status = 'DELIVERED' alone would NOT use idx_city_status.